

SEMWEAVER: Provenance-Guided Refinement of LLM-Generated Vulnerability Checkers

ANONYMOUS AUTHOR(S)

LLM-generated static vulnerability checkers can warn on both vulnerable and fixed code, or remain silent on vulnerable code. We study automated post-synthesis refinement under a patch-local, paired-warning gate. SEMWEAVER supplies the editor with provenance-labelled patch context, analyzer outputs, and analyzer-internal records: backend-produced intermediate representations bound to raw output and source scope. A candidate is adopted only after independent paired execution confirms a vulnerable-side warning and fixed-side silence.

We evaluate 39 frozen, generated-only KNighter CSA checkers: 15 initially warn on both revisions and 24 are silent on the vulnerable revision. KNighter’s actual refinement process yields one patch-discriminating success (PDS) and patch-version $F_1 = 0.441$; SEMWEAVER with the same GPT-6-Luna model yields seven PDS and $F_1 = 0.556$. In the directly matched 15-case fixed-noisy task, SEMWEAVER retains vulnerable-side warning signals and reduces fixed-side object alerts to 60 versus KNighter’s 72. Withholding strictly internal records leaves the full-39 outcome unchanged, so this study does not establish a general causal benefit from those records. Runs with GLM-5.3 and GLM-5.3-Flash also improve paired-version discrimination. These results support bounded, automatic reduction of patch-related warning noise and recovery of vulnerable-side warning signals, not independently adjudicated target-vulnerability recall or unseen-site generalization.

CCS Concepts: • **Software and its engineering** → **Automated static analysis**; • **Security and privacy** → **Systems security**.

Additional Key Words and Phrases: static analysis, vulnerability checkers, large language models, patch-guided refinement

1 INTRODUCTION

Static analysis is a cornerstone of software security assurance for systems code. Tools such as CodeQL [17] and Clang Static Analyzer (CSA) [31] detect vulnerabilities by checking source code against predefined rules without executing it [4, 7, 14, 41]. The effectiveness of these tools depends on the quality of their vulnerability checkers, each of which must encode the semantic conditions (path guards, state invariants, data-flow relations, and API contracts) that distinguish a real bug from safe code. Writing such checkers by hand requires deep expertise in both the target codebase and the analysis framework, making manual authoring expensive, slow, and difficult to scale.

LLM-based patch-driven synthesis offers a promising alternative. In this paradigm, a security patch serves as a compact before/after specification: the LLM consumes the patch and synthesizes a reusable checker that should fire on the vulnerable revision but remain silent on the fixed revision. KNighter [48], a representative system, has demonstrated that this approach can produce compilable and executable CSA checkers from historical Linux kernel fixes.

However, the resulting checkers may be semantically inadequate. A security patch is inherently underspecified: a one-line goto-target change may encode an ownership barrier, a widened cast may encode a range guard, and a moved free may encode a lifetime invariant. A checker that captures only the syntactic shape of the edit, without encoding the underlying semantic condition, can fire on both the vulnerable and fixed revisions. After naïve refinement, the checker may instead lose the vulnerable-side trigger entirely, a failure mode we call over-pruning. These outcomes can arise from incomplete API models, incorrect state updates, or a missing relation between the patch and the detector’s trigger.

Current refinement strategies, including KNighter’s own iterative loop, attempt to address this problem by operating at the output level [34, 44, 48]. They run the checker on both revisions, observe whether it fires on the wrong side, and feed this behavioral signal back to the LLM. The observed warning identifies a failed behavior, but may leave several plausible causes: a missing path guard, cleanup barrier, ownership transition, or API condition. An editor can reason from its patch and source context; additional analyzer representations provide another inspectable source of relevant structure.

We make a narrower observation: **some patch-relevant program structure is available in analyzer-produced intermediate representations** but is not ordinarily surfaced to the refinement loop. Path-sensitive analyzers such as CSA internally track branch guards, symbolic states, memory regions, and lifetime events during checker execution [31]. Relational query engines such as CodeQL maintain databases of data-flow edges, call-graph relations, type hierarchies, and API-usage patterns [2, 17]. These representations can suggest candidate predicates, but a CFG edge or call relation is not by itself a proof of path feasibility or checker correctness.

Exploiting these intermediate results, however, is not straightforward. Analyzer dumps and database relations can be much larger than the relevant patch scope. Selecting a bounded view also requires preserving its source binding and distinguishing backend-produced structure from diagnostic or source-derived context. The challenge is to select and present only the evidence that is relevant to the particular semantic gap in the checker under refinement.

SEMWEAVER provides post-synthesis checker editing guided by provenance-labelled analyzer context. SEMWEAVER is designed as a plug-in post-generation layer that is independent of the upstream LLM generator, treating any generated checker (from KNighter or another pipeline) as an artifact to be refined rather than regenerated. The framework addresses the evidence-selection challenge through three mechanisms. First, a rule-guided preflight extracts patch facts (e.g., guards, type widening, and affected calls) and prioritizes evidence requirements by inferred mechanism. The editor may request role views in the general workflow; the strict CSA comparison instead freezes its patch-scoped view before editing. For instance, a cleanup-edge patch triggers collection of path-guard and lifetime evidence from CSA, while a bounds patch triggers guard, flow, and API-usage evidence from CodeQL. Second, backend outputs, patch facts, source context, and validation feedback are normalized into a compact bundle with typed records, giving the editor a structured edit target (“add the patch-introduced cleanup barrier while preserving the freed-object transition”) instead of a vague behavioral symptom (“fixed side still warns”). Third, a strict paired-warning gate accepts a refinement only when the refined checker both warns on the vulnerable revision and eliminates fixed-side warnings, rejecting edits that silence both sides. This operational gate does not independently establish that a warning identifies the target vulnerability.

We evaluate SEMWEAVER on 39 frozen, generated-only KNighter CSA checkers that fail patch discrimination before refinement. Fifteen warn on both revisions; 24 are silent on the vulnerable revision. The complete system comparison runs KNighter’s actual refinement process and SEMWEAVER from the same starting artifacts. The directly comparable fixed-noisy task is the 15-case stratum, for which we additionally match the GPT model, fixed reports, and each case’s model-call ceiling. SEMWEAVER retains 60 fixed-side object alerts versus KNighter’s 72 while preserving vulnerable-side warning signals on all 15 cases. On all 39 cases, its GPT run reaches seven patch-discriminating successes (PDS) versus KNighter’s one, although the 24 no-report cases exercise a recovery capability absent from KNighter’s actual loop. A full-cohort ablation is tied when strictly internal records are withheld; we therefore do not attribute the system-level gain to those records alone. Separate automatic CodeQL and multi-model runs examine backend feasibility and the stability of patch-local refinement.

This paper makes the following contributions:

- We distinguish output-level behavioral feedback from provenance-bound analyzer-internal evidence, and formulate paired-warning refinement for vulnerability checkers generated by LLMs.
- We present SEMWEAVER, a plug-in refinement layer that organizes provenance-labelled analyzer representations, analyzer outputs, patch and source context, and validation feedback into a compact semantic evidence bundle; the strict CSA comparison freezes the available view rather than evaluating adaptive routing.
- We provide an automatic, denominator-complete 39-checker comparison with KNighter’s actual loop; distinguish same-task fixed-side noise reduction from vulnerable-side warning recovery; and report a full-cohort ablation, repeated-subset variability, and cross-backend limitations without pooling incompatible evidence.

2 BACKGROUND

This section introduces the background on static vulnerability checkers, LLM-based checker generation from security patches, and the analyzer-internal intermediate results that motivate our approach.

2.1 Static Vulnerability Checkers

Static analysis examines source code without executing it, aiming to find bugs, security vulnerabilities, and code-quality issues at compile time or during code review [4, 14]. A vulnerability checker is a rule or query within a static-analysis framework that flags program locations matching a security-relevant condition. Modern frameworks support different checker paradigms. Path-sensitive engines such as the Clang Static Analyzer (CSA) perform symbolic exploration over program paths, tracking branch guards, symbolic values, memory regions, and lifetime transitions to detect bugs like use-after-free, null dereference, and resource leaks [31]. Relational query engines such as CodeQL compile source code into a queryable database and express checkers as declarative queries over data-flow edges, call-graph relations, type hierarchies, and API-usage patterns [2, 17]. Both paradigms have been deployed at industrial scale for continuous vulnerability detection [7, 41].

Writing effective checkers remains a bottleneck. A checker must encode not only the syntactic pattern of a vulnerability but also the semantic conditions (path guards, state invariants, data-flow relations, and API contracts) that distinguish real bugs from safe code. This requires deep domain knowledge of both the target codebase and the analysis framework, making manual checker authoring expensive and error-prone.

2.2 LLM-Based Patch-Driven Checker Generation

Recent work has proposed using LLMs to automate checker generation from security patches. A security patch provides a compact before/after specification: the vulnerable revision should trigger the generated checker, and the fixed revision should not. This idea builds on a well-established line of work that treats patches as vulnerability specifications, including semantic patching [37], clone-based vulnerability discovery [19, 24], and patch-derived datasets [15]. KNighter [48] is a representative system that synthesizes CSA checkers from historical Linux kernel fixes and validates them against the vulnerable and fixed revisions.

However, the generated checkers are seldom semantically adequate. A patch is inherently underspecified: a one-line edit may encode a path guard, an ownership transition, a lifetime barrier, or an API contract, yet the generated checker often captures only the syntactic shape of the edit without the semantic condition the edit enforces. Existing refinement loops, including KNighter’s own iterative strategy, operate at the output level [34, 44, 48]. They feed behavioral signals (e.g.,

“the fixed side still warns”) back to the LLM, revealing the symptom but not diagnosing the cause. This feedback cannot tell the LLM whether the missing predicate is a path guard, a cleanup barrier, an ownership transition, or an API contract, so the model must guess which semantic condition to add.

2.3 Analyzer-Internal Intermediate Results

Static-analysis backends compute intermediate representations that can be exposed through debugging interfaces or database queries, but these are not normally supplied to a checker-refinement loop.

CSA, as a path-sensitive engine, internally maintains per-path symbolic states that record branch conditions, memory region bindings, allocation/free events, and object lifetime transitions [31]. CodeQL, as a relational engine, materializes a full program database from which data-flow edges, call-graph relations, type hierarchies, member-access patterns, and API-usage sequences can be queried [2, 17]. These intermediate facts have long underpinned taint analysis, graph-based detection, and interprocedural security reasoning [1, 21, 30, 46].

These representations may suggest a missing predicate, but a CFG branch or database tuple does not itself establish vulnerability semantics. The operational distinction between a backend-produced representation, a diagnostic, and a source-window fallback is central to our evidence accounting (Section 4.2). This motivates extracting bounded, provenance-labelled context for refinement while retaining paired validation as the correctness gate.

3 MOTIVATING EXAMPLE

This section traces an automatic refinement of the KNighter-derived CSA checker for Linux commit 97cba232549b, which repairs an off-by-one access in the AMD display driver [43, 48]. The example has an explicit safety relation: the largest look-ahead index must be smaller than the extent of the array actually accessed. Both the starting and refined checkers use `checkASTCodeBody`; the edit adds a missing bound relation to the existing analysis.

The bug and its patch. The member `dc->links` has declared capacity `MAX_PIPES * 2` (Figure 1, ❶). We abbreviate this constant as N in the figure; this is a presentation alias, not a new program variable. The function `get_host_router_total_dp_tunnel_bw` iterates over candidate links and, on a matching host-router branch, reads both `links[i]` and `links[i + 1]` (M10–M11, ❸). With the original condition $i < N$ (M5, ❷), the loop permits $i = N - 1$, so the look-ahead can reach index N , outside the array. The patch changes the bound to $i < N - 1$ (M6, ❹), which limits the largest look-ahead index to $N - 1$. Other branch conditions determine whether an access occurs, but the patched loop bound already prevents this final iteration from reaching an out-of-range look-ahead.

Why the baseline checker fails. The generated checker already recognizes an induction variable, a loop bound, and a look-ahead array subscript. It reports an access when it does not recognize a protective condition in an enclosing `if` (B1–B5, ❺). That rule overlooks a guard carried by the loop condition itself. The fixed program still contains `links[i + 1]`, so the checker reports on both revisions despite the reduced loop bound. The independent patch-related-object replay yields 1/1 alerts. The refinement problem is therefore to interpret the relation between the loop bound and array extent while retaining the unsafe access.

Evidence-guided refinement. The available bundle contains four provenance-labelled records: two strictly internal Clang 18 CFG/call-graph records and two analyzer-output records. The internal records anchor the affected loop and branch context to raw dumps from `debug.DumpCFG` and `debug.DumpCallGraph`; patch and source context identify the bound change. The declared array extent is read from the AST type by the emitted checker. It is not a path-feasibility proof supplied by the evidence bundle.

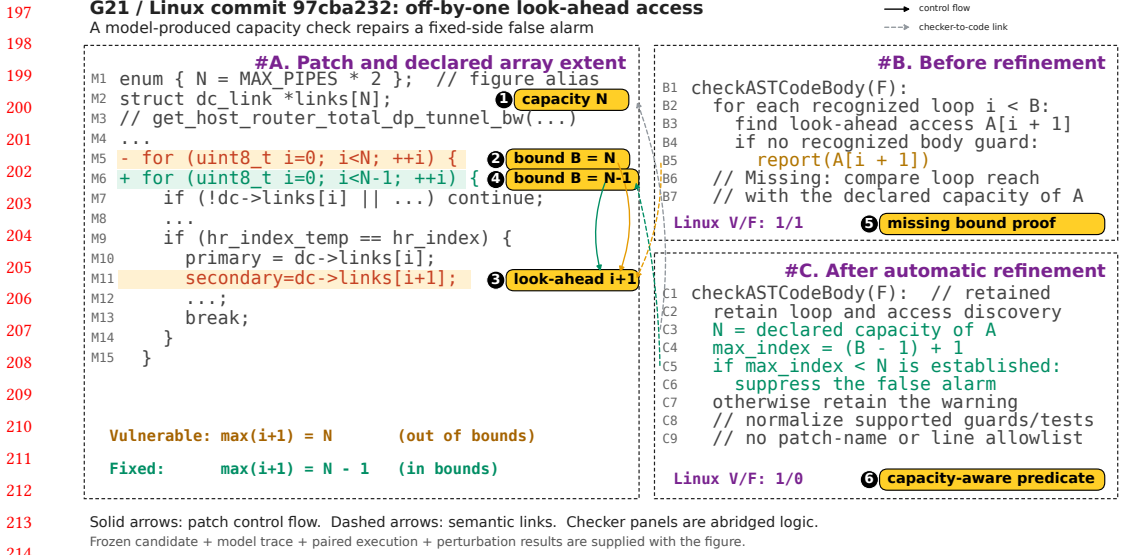


Fig. 1. **Capacity-aware look-ahead refinement.** Panel A combines an abridged array declaration and patch hunk; N denotes $\text{MAX_PIPES} * 2$. Panels B and C show abridged checker logic. The original checker reports 1/1 vulnerable/false alerts; the independently replayed model-produced refinement reports 1/0. Solid arrows show control flow, dashed arrows semantic correspondences. Orange marks the unsafe bound/access and teal the patched bound and capacity test.

The initial GLM-5.3-Flash edit adds a capacity-aware condition while retaining the original traversal. For the depicted loop $i < B$, the maximum look-ahead index is $(B - 1) + 1 = B$. A known extent N justifies suppressing the warning only when this index is smaller than N (C3–C6, ⑥). Consequently, the vulnerable $B = N$ remains reported and the fixed $B = N - 1$ becomes silent. This condition depends on type and index relations rather than the patched function name, a particular array capacity, or adjacency of the relevant statements. The initial edit uses two model calls and passes the independent Linux replay at 1/0.

Checking the example beyond its patch. Post-hoc development probes expose shortcomings in the initial candidate’s fallback and inherited guard recognition. Model-driven quality continuations receive these hash-bound diagnostics and repair the checker; the raw model edits and failed intermediate versions are retained. Four automatic GPT-6-Luna-high quality continuations use 12 additional model calls (14 including the initial Flash refinement). The final frozen checker retains 1/0 on the original Linux pair. It passes all 28 core fixtures, six boundary fixtures, 20 additional guard/bound fixtures, and 60 renamed/capacity-varied executions of those 20 forms: 58/58 unsafe and 56/56 safe executions overall. The labels for the latter 80 executions were also checked with AddressSanitizer. These are finite development diagnostics, not 114 independent bugs. The fixtures and diagnostic feedback are developer-authored; every checker-code edit is model-produced. The tests support robustness to the tested name changes, unrelated statements, constant capacities, and equivalent unit-step-loop/guard forms. Arbitrary aliasing, dynamic extents, and non-unit loop updates remain outside this claim. This illustrative continuation is separate from the frozen 39-case results and does not establish a causal benefit from internal records alone.

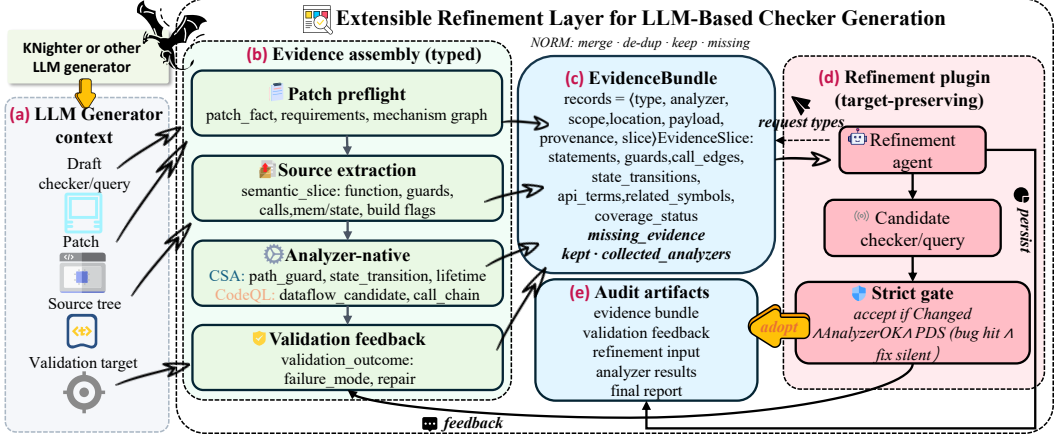


Fig. 2. **SEMWEAVER architecture.** (a) An LLM generator produces a draft detector from the patch. (b) A rule-guided patch preflight plans evidence requirements and four collectors gather corresponding facts; the figure’s “analyzer-native” collector label does not mean every emitted record has internal provenance. (c) Facts are normalized into a typed EvidenceBundle with seven persisted record types. (d) The refinement agent edits the detector using role-specific views. The gate’s “bug hit” denotes vulnerable-version warning presence, not independently verified target detection. It requires that signal and fixed-side silence; rejected candidates feed back into the bundle (dashed arrow). (e) Adopted candidates, evidence bundles, and validation logs are persisted as audit artifacts for reproducibility.

4 APPROACH

SEMWEAVER is a post-synthesis refinement layer for LLM-generated vulnerability detectors. Given a draft CSA checker or CodeQL query produced by an upstream generator, the patch that motivated the synthesis, and the corresponding source tree and validation scope, SEMWEAVER refines the detector to encode missing semantic predicates while retaining a vulnerable-side warning signal.

Figure 2 shows five stages. **(a) Generator context.** SEMWEAVER receives a draft detector from an upstream LLM generator (e.g., KNighter), together with the patch, source tree, and validation target. It treats the generator as a black box: any synthesized detector is an artifact to be refined, not regenerated. **(b) Evidence assembly.** Rule-guided patch preflight infers a likely mechanism from patch facts and available analysis metadata, then prioritizes evidence requirements. Four collectors (patch preflight, source extraction, analyzer representations or outputs from CSA/CodeQL, and validation feedback) gather evidence (Section 4.1). **(c) Evidence bundle.** All facts are normalized into a typed EvidenceBundle with seven persisted record types, giving the refinement agent structured edit targets rather than raw analyzer dumps (Sections 4.2 and 4.3). **(d) Paired-warning refinement.** The refinement agent edits the detector using role-specific evidence views from the bundle. A strict gate accepts a candidate only when it preserves a vulnerable-side warning signal and eliminates fixed-side warnings. Rejected candidates re-enter the bundle as feedback for the next iteration (Section 4.4). **(e) Audit artifacts.** Each run persists the starting and candidate detector, evidence bundle, prompt/response trace, validation logs, and final report. The verifier recomputes raw-output hashes, checks the recorded Clang interface/schema and patch file/function binding for every internal record, recounts provenance classes, and then checks the compiled candidate against paired vulnerable/fixed logs. It does not infer evidence origin from an LLM explanation or rerun a model to reproduce a deterministic count.

Algorithm 1 Paired-Warning Evidence-Guided Refinement

Require: detector D , patch P , source S , scope V , backends A , budget K
Ensure: refined detector D^* and audit report

```

1:  $R_0 \leftarrow \text{VALIDATE}(D, P, V)$ 
2:  $E_p, E_s \leftarrow \text{PREFLIGHT}(P), \text{EXTRACTSOURCE}(P, S, D)$ 
3:  $E_a \leftarrow \bigcup_{a \in A} \text{COLLECTNATIVE}(a, D, P, S)$     $E_v \leftarrow \text{FEEDBACK}(R_0)$ 
4:  $B \leftarrow \text{NORMALIZEMERGE}(E_p, E_s, E_a, E_v)$ 
5: if  $\text{PDS}(D, P, V)$  then
6:   return  $\text{AUDIT}(D, B)$ 
7: end if
8:  $D^* \leftarrow D$ 
9: for  $i = 1$  to  $K$  do
10:   $V_i \leftarrow \text{AGENTSELECTVIEW}(B)$     $C_i \leftarrow \text{AGENTEDIT}(D^*, P, V_i)$ 
11:   $R_i \leftarrow \text{VALIDATE}(C_i, P, V)$ 
12:  if  $\text{CHANGED}(C_i, D^*) \wedge \text{ANALYZEROK}(R_i) \wedge \text{PDS}(C_i, P, V)$  then
13:     $D^* \leftarrow C_i$    break
14:  else
15:     $E_i \leftarrow \text{FEEDBACK}(R_i)$     $B \leftarrow \text{MERGE}(B, E_i)$ 
16:  end if
17: end for
18: return  $\text{AUDIT}(D^*, B)$ 

```

4.1 Evidence Selection and Exposure

The central design choice in evidence assembly (Figure 2b) is *what to collect and expose*. Analyzer outputs can be large and heterogeneous; passing them wholesale to the editor obscures patch-relevant relations. The preflight planner represents candidate requirements by role and priority, while source/function scope and bounded views limit what the editor sees. In the strict CSA comparison, collection is precomputed and the visible view is frozen, so its result does not measure demand-driven routing.

Rule-guided preflight and role requests. The implementation has no separately trained or prompted patch-mechanism classifier. PatchFactsExtractor parses the patch and metadata for affected functions, added guards, risky calls, and type widenings. EvidencePlanner takes a primary pattern from analysis metadata when available or infers it from those facts, then assigns fixed priorities to requirements: patch anchoring is 100, lifetime/state requirements for double-free or use-after-free are 92/89, and a guard for null or bounds cases is 90. It sorts by descending priority with evidence type as a tie-breaker and retains at most eight planned requirements in the default configuration. Table 1 summarizes mechanism-to-role examples; it is not a classifier confusion matrix or a promise that every role is analyzer-internal. The normalizer does not learn or re-rank facts. In the general workflow the editor may request evidence types, with duplicate and request-count limits. For the strict provenance-audited CSA comparison, patch-scoped records are precomputed. The treatment preloads up to the first four distinct strictly internal record types in persisted order and disables additional requests; its no-internal counterpart sees no internal records but retains patch, source, and feedback context. This deterministic exposure policy is a limitation, not an adaptive classifier. The matched result tests the frozen native-evidence view, *not* a routing policy’s accuracy or benefit.

4.2 Evidence Sources and Roles

Given the patch scope and any planned requirements, four collectors execute in sequence (Figure 2b, top to bottom):

Table 1. Rule-guided evidence requirements by patch mechanism (representative); names are persisted record types, not gold labels or measured classifier outputs.

Patch mechanism	Possible record types	Backend
Cleanup edge / double free	allocation_lifecycle, state_transition, path_guard	CSA
Bounds widening / overflow	semantic_slice, dataflow_candidate, state_transition	CSA/CodeQL
Initialization / null deref	state_transition, path_guard	CSA
Use-after-free / escape	allocation_lifecycle, call_chain	CSA/CodeQL
API misuse / missing check	semantic_slice, call_chain	CodeQL

Table 2. Evidence primitives and provenance in the audited 39-case CSA cohort. One record may contain several CFG primitives; counts are by record origin, not primitive.

Origin	What the persisted record contains	Records
Analyzer-internal	Clang CFG branches/guards, value bindings, and call-graph edges, bound to raw dump and function	80
Analyzer output	CSA diagnostics and paired validation reports; no internal state	51
Source-derived	Patch facts, source windows, or compile-database-assisted slices	61

Patch preflight identifies changed files, hunks, and statements, and produces a mechanism graph that links the edited code to likely bug patterns. This supplies the structural skeleton: *what changed* and *where*.

Source extraction resolves patch files and records bounded contexts around the changed code, including enclosing functions, branch guards, callee signatures, memory/state operations, and build flags. This provides local semantic context without dumping repository-scale source.

Analyzer evidence is classified by its actual origin, not by the collector’s name. We call a record *analyzer-internal* only if its payload is derived from a backend-produced intermediate representation, identifies the interface and raw output, and is bound to a source revision, file, and function. In the measured CSA cohort the interface is Clang 18’s debug.DumpCFG and debug.DumpCallGraph; the persisted UTF-8 dump is parsed into schema `semweaver.csa_cfg_snapshot.v1`, with a raw-output SHA-256 and source-scope binding. These dumps provide CFG branches and value bindings and call edges. They do *not* themselves prove a path-feasible lifetime or ownership transition. Analyzer diagnostics and alert counts are *analyzer output*, not internal state. Patch diffs, source windows, and compile-database-assisted source extraction are *source-derived* fallbacks even when a CSA or CodeQL collector emits them. A live CodeQL database query could meet the internal-evidence definition if its query, database revision, and tuples were retained; we do not attribute source-window-derived CodeQL facts to that category.

This distinction makes extraction challenging in a practical sense: backend intermediates are verbose, backend-specific, sometimes unavailable for the patch-related translation unit, and must be linked to the same revision and function as the checker failure. The collector retains missing-evidence markers rather than silently upgrading fallback source text to internal evidence. Across all 39 frozen CSA evidence bundles, 38 contain at least one strictly internal record and one abstains. Of 192 emitted records, 80 are analyzer-internal, 51 are analyzer outputs, and 61 are source-derived. Every internal record passes the raw-output hash and patch-scope binding audit. These counts characterize evidence availability, not refinement success or CodeQL-native coverage.

Validation feedback records whether the baseline detector compiles, executes, warns on the vulnerable revision, remains fixed-side noisy, or loses that warning signal. This provides the failure *symptom*, while the preceding three sources supply the semantic *edit target*.

The persisted type vocabulary is `patch_fact`, `semantic_slice`, `dataflow_candidate`, `call_chain`, `path_guard`, `allocation_lifecycle`, and `state_transition`. A `patch_fact` anchors changed statements; `semantic_slice` summarizes bounded statements, guards, API terms, and related symbols. `dataflow_candidate` and `call_chain` describe possible propagation and call context, while `path_guard`, `allocation_lifecycle`, and `state_transition` describe control, resource, and abstract-state context. “Barrier,” “type filter,” and “API usage” are semantic interpretations inside these records, not additional persisted types or claims of proven path feasibility.

When a selected role yields no evidence, the bundle retains an explicit missing-evidence marker rather than treating the absence silently, so the refinement agent knows what was requested but unavailable.

4.3 Evidence Bundle Schema

The normalizer merges all collected facts into the EvidenceBundle shown in Figure 2c. Each fact becomes an EvidenceRecord carrying a type, the contributing analyzer, scope, source location, payload, provenance, and an optional EvidenceSlice (the compact semantic content consumed by the agent). The schema separates program facts from analysis rules in the spirit of relational analyzer frameworks [2, 5, 20], but uses a smaller role vocabulary tailored to checker refinement. Formally:

$$\begin{aligned}
 B &= \text{Norm}(E_p \cup E_s \cup E_a \cup E_v), \\
 E_a &= E_{\text{CSA}} \cup E_{\text{CodeQL}}, \\
 r \in B.\text{records} &= \langle \text{type}, \text{analyzer}, \text{scope}, \\
 &\quad \text{location}, \text{payload}, \text{provenance}, \text{slice} \rangle.
 \end{aligned} \tag{1}$$

Here E_p contains patch facts, E_s bounded source slices, E_a analyzer-internal representations or outputs filtered by selected roles (Section 4.1), and E_v validation outcomes. Each record’s provenance has one of three origins: `analyzer_internal`, `analyzer_output`, or `source_derived`. The normalizer deduplicates records and marks missing evidence; origin is neither a confidence score nor a proof of semantic correctness.

The type-to-backend mapping reflects available interfaces: CSA CFG/call-graph dumps inform guard, state, and call-chain records; CodeQL queries can support flow and relation records when a live database is available. These are record types, not seven distinct backend analyses. Cross-function call edges provide context, but the present CSA dump does not establish an interprocedural path or ownership proof. Provenance remains visible when a type is populated only by a fallback.

The bundle is bounded by design: preflight requirements and collectors restrict scope to patch-related files and validation targets, while the editor receives selected or preloaded role views rather than the full raw dump (Figure 2d). These constraints control prompt size and allow the contribution of strictly internal records to be tested by a matched ablation. A small, single-run ablation cannot by itself establish their causal contribution across the cohort.

4.4 Paired-Warning Refinement

The refinement loop (Figure 2d) receives the current detector, patch, baseline validation notes, and a bounded role-specific evidence view. The interface supports requesting further roles from the persisted bundle (“request types” arrow), but the provenance-audited matched condition freezes the visible view before the edit and disables additional agent requests. Thus its measured effect

is a contrast between preselected native-evidence and no-internal views, not a test of dynamic evidence retrieval. This keeps the prompt bounded and the two conditions auditable.

The agent’s tool-mediated loop draws on recent LLM agent and self-refinement architectures but constrains actions to attached evidence, detector edits, and validation commands [34, 42, 44, 50]. Where evidence requests are enabled, they replay attached records rather than launch open-ended repository exploration; the request count is capped and unused or missing evidence is logged. In the matched condition, request count is zero by protocol.

Model versus deterministic components. LLMs may produce patch-analysis metadata and the editor’s checker edit; when enabled, the editor also chooses which evidence types to request. The preflight requirement rules, raw-output hashing and scope binding, record normalization, checker compilation, paired replay, and acceptance are programmatic. The LLM does not create a backend dump, set a provenance origin, or decide PDS. The audit retains the exact starting checker, prompt and response, visible roles, raw evidence, candidate source, and validation decision. This boundary prevents an LLM explanation from being counted as analyzer-produced evidence. The editor prompt supplies the current and starting checker paths, patch path, bounded source/evidence view, baseline vulnerable/fixed symptom, backend identity, and edit/validation rules. It asks for an in-place checker edit that models the mechanism rather than matching patch spellings, forbids suppressing the vulnerable trigger, and requires compilation before paired acceptance. Compiler-repair prompts receive the actual build diagnostics; all available English templates and rendered exchanges are named in the artifact manifest, with absent historical raw exchanges explicitly marked rather than reconstructed.

Strict gate. SEMWEAVER treats validation as an analyzer-facing and paired-warning gate (Figure 2d). A candidate must (1) change the detector, (2) compile and execute under its backend harness (ANALYZEROK), and (3) achieve PDS: a vulnerable-revision warning and fixed-revision silence. Silencing both sides is rejected as signal loss. “Target-preserving” in the architecture denotes this operational signal constraint, not proof that the target vulnerability remains detected. Additional code variants and target-location correctness require separate validation.

Feedback loop. When the gate rejects a candidate, the validation outcome is fed back into the bundle as new evidence (feedback arrow in Figure 2), enriching subsequent refinement attempts with the specific failure mode (e.g., “lost vulnerable-side trigger at line X”). The loop terminates when a candidate passes the gate or the budget K is exhausted (Algorithm 1). The direct 15-case call-matched comparison freezes the evidence view and allows one model call before each external vulnerable/fixed replay. A subsequent permitted call receives the preceding build or paired-alert feedback, bound to its candidate hash. The first PDS candidate ends the case; each call ceiling comes from KNighter’s actual calls on that subject. The separate full-cohort and repeated-subset conditions use the budgets stated in Section 5.

Backend extension surface. An adapter must provide registration, evidence/provenance mapping, dependencies, execution/result decoding, and paired validation. Shared modules still contain backend branches. Four enumerated CodeQL-specific modules occupy 2,438 physical lines, including comments and generation code: an existing footprint, not code to rewrite for each backend. With a usable analyzer and patch pair, we estimate that a familiar developer using Codex can complete basic adaptation within an afternoon by reusing these interfaces. This is a planning estimate, not measured labor or a scalability result. New native-evidence semantics and broad validation require separate work. The artifact lists prerequisites and tasks. A new bug mechanism needs a role mapping and safe/unsafe fixtures, and a new extractor only for unavailable relations.

5 EVALUATION

We evaluate SEMWEAVER’s patch-related false-positive reduction and its sensitivity to verified analyzer-internal records. The scope is paired patch revisions and related build objects, not whole-kernel bug discovery.

5.1 Research Questions

RQ1: Generated CSA checkers. On the same frozen generated checkers, how do KNighter’s actual refinement process and SEMWEAVER affect patch discrimination, vulnerable-side hits, and fixed-side noise?

RQ2: Cross-backend feasibility and provenance. Which detector formats can the layer process, and what can the historical independent-generator data support once manual construction and missing lineage are accounted for?

RQ3: Evidence sensitivity. What changes when strictly analyzer-internal records are withheld under an otherwise matched feedback protocol?

RQ4: Model robustness. Do automatic patch-local gains recur with different editor models and across repeated decodes of a frozen subset?

5.2 Subjects

E1 freezes one automatically generated CSA checker per selected Linux commit from the upstream KNighter artifacts. The inventory has 286 generated records on 60 commits; 241 records on 39 commits have no upstream refinement-status row. From these, a frozen rule selects one per commit by upstream TP, then TN, then lowest checker index, taking the nonempty repaired checker source. This non-random ranking defines inputs only: its old scores are never used as our outcome labels. We materialize and independently replay all 39 against vulnerable and fixed revisions before treatment. All fail initial PDS: 15 warn on both revisions, while 24 are silent on both. These operational warning strata define different edit tasks, not independently adjudicated target detections or misses. No manually completed checker or human repair enters any effectiveness numerator.

Both KNighter’s actual report-driven loop and SEMWEAVER start from the same 39 generated checkers. In the 15 fixed-noisy cases, both have a report to refine; this is the direct noise-reduction comparison. In the 24 no-report cases, KNighter’s actual branch compiles but makes no model edit, whereas SEMWEAVER tries warning recovery. The 39-case comparison is therefore a complete system comparison, not a claim that KNighter unsuccessfully attempted 24 recovery edits. KNighter does not produce or refine CodeQL queries in this cohort.

E2 is a separate automatic CodeQL feasibility replay using an archived query from an independent generator. Historical rows whose construction or lineage was manual, skipped, or ambiguous are excluded from automatic effect counts. CodeQL is not pooled with the 39 CSA cases. E3 compares native and no-internal SEMWEAVER arms on all 39 cases with GPT-6-Luna high and an eight-call ceiling; it also repeats a hash-selected subset of five fixed-noisy and seven initially silent cases three times per arm. E4 runs three model configurations on the full 39 under a two-call ceiling and repeats the same 12-case subset. Repeated decodes reuse subjects and are not additional independent checkers.

There is no separate LLM mechanism classifier to score. Fixed rules map patch and analysis facts to prioritized evidence requirements; the editor may request roles in the general workflow. The strict CSA comparison freezes this view and disables such requests. We neither report classifier accuracy nor claim the fixed priorities are optimal. A routing-sensitivity study would hold checker, model, and budget fixed while perturbing visible roles; the historical five-case ablation is not such a study.

Table 3. Evaluation terms and acceptance criteria.

Term	Definition	Success?	Why
Vulnerable positive	checker warns on the vulnerable version	needed	warning signal
Fixed silent	no checker alert in fixed-side scope	not alone	may over-prune
PDS	vulnerable positive $\wedge$ fixed silent	yes	version discrimination
Signal loss	fixed silent but vulnerable warning lost	no	over-pruning
Fixed-noisy	warnings on both versions	no	residual noise

5.3 Metrics

The primary metric is *patch-discriminating success* (PDS): a detector must emit a warning on the vulnerable revision and none on the fixed revision within the same patch-related scope. This is a version-level warning criterion, not a target-location oracle. We write PDS over a detector D , patch P , and validation scope V :

$$\begin{aligned}
 H_v(D, P, V) &= \#[\text{alerts}(D, V_{\text{vuln}}, P) > 0], \\
 S_f(D, P, V) &= \#[\text{alerts}(D, V_{\text{fix}}, P) = 0], \\
 \text{PDS}(D, P, V) &= H_v(D, P, V) \wedge S_f(D, P, V).
 \end{aligned} \tag{2}$$

We report both components: *vulnerable positive* for a warning on the vulnerable side, and *fixed silent* for silence under the same fixed-side scope. A detector that silences both sides loses the vulnerable warning signal and is not a success. Table 3 summarizes the criteria.

The refinement gate accepts a candidate D' only when it changes the baseline detector, remains executable for the backend, and satisfies PDS:

$$\begin{aligned}
 \text{Accept}(D', D) &= \text{Changed}(D', D) \wedge \text{AnalyzerOK}(D') \\
 &\quad \wedge \text{PDS}(D', P, V).
 \end{aligned} \tag{3}$$

All conditions use the same PDS definition and track vulnerable-side loss. E1 counts come from object-level scan-build logs over patch-related build objects. Static artifact-review findings are retained with each candidate as a separate quality diagnostic. Compilable candidates are replayed on both revisions even when structural review raises a concern; the reported PDS metric is determined by the paired executions. The artifact records both the review and paired verdict.

For the full 39-checker portfolio, we additionally binarize each checker/revision pair by warning presence. Each vulnerable revision is one positive and its fixed revision one negative; thus precision, recall, and $F_1 = 2TP / (2TP + FP + FN)$ are computed over 78 *patch-version decisions*, not individual analyzer alerts or unseen vulnerability sites. Fixed-side alerts measure review burden, not an exhaustively adjudicated false-positive count. A vulnerable-side warning need not identify the patch's target: G08 warns on allocation failure rather than the registration-failure leak repaired by its patch. The artifact records qualitative diagnostic/patch correspondence, but we do not claim exhaustive target-level recall. Failed analyzer executions are unscored rather than converted into false negatives.

5.4 Experimental Environment

CSA runs in a pinned Clang/LLVM 18.1.3 and Python 3.12.3 container with scan-build; the automatic CodeQL replay uses CodeQL 2.26.4 with codeql/cpp-all 8.0.3 and no-build C/C++ databases. Each Linux subject uses its recorded patch commit and parent with allyesconfig, not a common release snapshot. Each starting, baseline-refined, and candidate checker is replayed on

the same patch-related build-object scope. We retain frozen manifests, source and checker hashes, raw analyzer outputs, model exchanges, validation logs, and adoption decisions. A provider or execution interruption is not converted into a checker outcome.

Use of generative AI in the research workflow. LLMs generated the starting checkers and edit candidates; patch-fact extraction, evidence-origin checks, normalization, validation, and adoption are programmatic. An AI coding assistant (Codex) assisted with experiment-driver implementation, audit/analysis scripts, and manuscript drafting. It supplied no ground-truth PDS labels: pinned source, compilation, paired analyzer runs, and the paired-warning rule determine reported outcomes. The artifact retains model settings, English prompts, exchanges, candidates, and script digests; a floating model alias is not an immutable model revision. Authors remain responsible for verifying AI-assisted code and interpretations.

5.5 Baselines and Variants

The unmodified checker is the shared before/after reference. E1 runs KNightier’s actual output-feedback refinement loop and SEMWEAVER from that same checker. The complete 39-case system comparison uses GPT-6-Luna at high reasoning effort. Its direct 15-case fixed-noisy comparison additionally shares the first five frozen fixed-side reports (or all when fewer) and gives each SEMWEAVER case at most the number of model calls KNightier actually used for that case; all candidates receive independent paired replay. SEMWEAVER may stop earlier. On the 24 no-report cases we compare system outcomes but do not describe KNightier as a recovery-method baseline.

Only the native versus no-internal contrast within SEMWEAVER probes access to strictly internal records. The native arm receives hash-bound Clang CFG and call-graph records; the other withholds them but keeps patch, source, analyzer-output feedback, model, budgets, and validator. The all-39 ablation uses eight calls per case; the separate matched 15-case comparison uses KNightier-derived per-case ceilings. Historical five-case, manual, and mixed-provenance contrasts are not evidence of a native-record effect.

E4 uses GPT-6-Luna high, GLM-5.3, and GLM-5.3-Flash to examine whether useful refinements recur across editor models. Each condition retains its named model across all subjects. The full-cohort runs have a two-call ceiling per case. Per-call output ceilings are 16,384 tokens for GPT and 65,536 or 131,072 for GLM; the retained manifests identify each run’s ceiling and implementation. All GPT repeats retain two calls per subject. Both additional GLM repeats use the same frozen implementation and a 131,072-token output ceiling, with two calls for fixed-noisy subjects and eight for warning recovery. The first E4 decode reuses the two-call full-cohort run. These are configuration-specific robustness checks, not a budget-matched repeated model ranking; matching calls also does not match token expenditure.

5.6 Protocol

E1 counts checker warnings in scan-build outputs on patch-related objects. We run KNightier’s report triage, repair, compiler repair, and validation loop, then independently replay its emitted checker. Each SEMWEAVER attempt is compiled; compilable candidates are replayed on both patch revisions before feedback informs a later call. Only PDS candidates are adopted. A 39-case portfolio retains the original checker when no candidate passes the gate. Compile failures, validity losses, residual noise, provider interruptions, and paired-execution failures have distinct recorded outcomes. Interrupted suffixes use fresh manifests; completed prefixes remain unduplicated. The artifact includes English templates, frozen inputs, exchanges, raw outputs, provenance bindings, and adoption decisions.

Table 4. Complete 39-case CSA system comparison with GPT-6-Luna high. F_1 uses 78 patch-version decisions; alerts count fixed-side patch-related objects. The 24 initially silent cases exercise recovery only in SEMWEAVER.

Condition	PDS	TP	FP	FN	F_1	Fixed alerts
Unmodified	0	15	15	24	.435	73
KNighter loop	1	15	14	24	.441	72
SEMWEAVER	7	20	13	19	.556	71

Table 5. Direct 15-case fixed-noisy comparison. Starting checkers warn on all vulnerable revisions and have 73 fixed-side object alerts. Calls are actual; SEMWEAVER uses no more than KNighter’s per-case calls.

Method	PDS / 15	Fixed alerts	Model calls
KNighter loop	1	72	99
SEMWEAVER, native	3	60	94
SEMWEAVER, no internal	2	71	97

6 RESULTS

6.1 RQ1: Generated CSA Checkers

Table 4 reports every frozen checker under adopt-only-on-PDS scoring. Initially, 15 checkers warn on both revisions and 24 are silent on both. This 0/39 PDS is a starting state, *not* KNighter’s refinement result. KNighter’s actual loop reaches one PDS; SEMWEAVER with GPT-6-Luna high reaches seven, comprising two fixed-noisy refinements and five recoveries from vulnerable-side silence. Patch-version F_1 rises from 0.435 for the unmodified cohort to 0.441 for KNighter and 0.556 for SEMWEAVER. Fixed-side object alerts fall from 73 to 72 and 71, respectively. Neither adopted portfolio loses a previously present vulnerable-side warning signal; this is not an exhaustive target-location check.

The 15 fixed-noisy cases permit the closer same-task comparison in Table 5. All three arms start from the same generated checkers and frozen reports, use GPT-6-Luna high, and share KNighter’s actual *per-case* model-call ceiling. KNighter reaches one PDS and retains 72 fixed-side alerts after 99 model calls. Native SEMWEAVER reaches three PDS and retains 60 after 94 calls; its no-internal control reaches two PDS and retains 71 after 97 calls. Every post-adoption portfolio remains warning-positive on all 15 vulnerable revisions. The 12-alert native-versus-KNighter gap measures reduced review burden on patch-related objects, not a whole-project false-positive rate. Fewer calls do not establish lower token or monetary cost. KNighter’s no-report branch makes no edit on the other 24 subjects.

6.2 RQ2: Cross-Backend Feasibility

We exclude the historical mixed-provenance 20-detector table from automatic-effect counts. In a separate development-only CodeQL replay, an archived automatically generated ImageMagick query starts at 0/0 vulnerable/fixed rows. Three independent model decodes of the same frozen input reach strict PDS twice (8/0 after one call each); the third recovers a warning signal but retains 22/22 rows after eight calls. The successful edit corrects a relative-path match while retaining exact file, function, and local-variable filters. This establishes patch-local operation on a second detector format, not cross-backend transfer or evidence causality. A second archived query already starts at 2/0 and is excluded from gain counting. KNighter is not a CodeQL baseline here.

Table 6. Full 39-case outcomes by editor model under the settings in Section 5. All rows share the frozen starting checkers and paired adoption rule.

Editor model	PDS / 39	F_1	Fixed alerts
GPT-6-Luna high	7	.556	71
GLM-5.3	11	.611	67
GLM-5.3-Flash	8	.575	71

Second-backend costs. On the same existing CodeQL databases, three paired replays per query retain the original 0/0 and refined 8/0 outcomes. Median query-plus-decode time is 12.57 s before and 36.51 s after refinement (two threads); peak process RSS is 976 and 1,078 MiB. Successful database-creation logs span approximately 31/30 s for the two revisions, excluding unlogged startup. The three original model decodes consume 15,122/14,345/110,282 tokens. These are one-case, existing-environment costs, not engineer-hours, cold-install latency, or a general scalability result.

6.3 RQ3: Internal-Evidence Sensitivity

All 39 evidence replays completed before treatment: 38 subjects had at least one strictly internal record, and one abstained. Of 192 persisted records, 80 are analyzer-internal, 51 are analyzer outputs, and 61 are source-derived; all 80 internal records pass raw-output and source-scope binding. The full-39 GPT ablation reaches *the same seven PDS cases* and $F_1 = 0.556$ with or without internal records. Each arm retains 71 fixed-side object alerts under PDS-only adoption. Thus the full-cohort gain cannot be attributed to access to internal records alone.

In the separate per-case-call-matched 15-case condition (Table 5), the native arm achieves one additional PDS and retains 11 fewer fixed alerts than no-internal. This difference is concentrated in G17, a concurrency checker with 11 starting fixed alerts. It has a different call ceiling and does not overturn the full-cohort tie. On the frozen 12-case repeat subset (five fixed-noisy and seven initially silent cases), native PDS totals are 2, 2, 3 and no-internal totals 1, 3, 2 (Table 7). Repeated draws on the same subjects do not establish a stable per-subject evidence effect.

6.4 RQ4: Model Robustness

Table 6 reports full-39 outcomes under a two-call ceiling. GLM-5.3’s eleven PDS comprise four fixed-noisy refinements and seven warning recoveries; GLM-5.3-Flash’s eight comprise two and six. Useful paired-version refinements occur with all three evaluated editor configurations. All rows use the same starting $F_1 = 0.435$ and 73 fixed-side object alerts.

Candidate traces also show changes short of PDS. For example, a Flash candidate for G03 changes vulnerable/fixed warnings from 2/2 to 8/6: vulnerable-side positivity alone does not ensure reduced noise. G14 instead reduces fixed-side alerts from three to one while remaining vulnerable-positive. Such non-PDS candidates remain diagnostic records and are *not adopted* into the strict F_1 or PDS totals. G08 further shows why a reduced count does not prove preservation of the target bug (Section 5).

Repeated subset. Table 7 makes both variation and the call-ceiling difference explicit. Additional GLM decodes allow eight recovery calls versus GPT’s two; their higher recovery totals therefore cannot establish model superiority. Fixed-noisy improvements are sparse and neither additional GLM decode attains one. These are descriptive configuration outcomes, not a controlled model ranking or independent-subject significance test.

Table 7. Three decodes on the same 12 subjects. Each cell is fixed-noisy PDS / initially-silent PDS, with denominators 5 and 7. E3 allows eight calls per subject. E4 GPT always allows two; GLM decodes 2–3 allow two for fixed-noisy and eight for recovery. E4 decode 1 reuses the two-call full-cohort run.

Condition	Decode 1	Decode 2	Decode 3
E3 native	0 / 2	0 / 2	1 / 2
E3 no internal	0 / 1	0 / 3	1 / 1
E4 GPT-6-Luna high	0 / 2	0 / 2	0 / 1
E4 GLM-5.3	1 / 2	0 / 4	0 / 5
E4 GLM-5.3-Flash	0 / 2	0 / 4	0 / 3

6.5 Qualitative Findings

The motivating example preserves the original AST-based traversal and adds an actual-capacity bound, followed by model-produced corrections to guard handling. Development probes vary names, unrelated statements, array capacities, and equivalent control flow. The full checker and failed intermediate candidates remain inspectable. A candidate that silences both revisions is rejected. Compile failures and execution-invalid attempts are not promoted to PDS; the original checker remains the portfolio member when no candidate passes the gate. Qualitative source checks link the five newly warning-positive GPT PDS cases to their patch mechanisms, but do not replace target-level adjudication across the cohort.

7 DISCUSSION

Scope of improvement. SEMWEAVER supports patch-related checking. It reduces fixed-side noise while retaining vulnerable-side warning signals on paired build objects; deployment on a changed fork or later version requires additional stability tests. Target-location correctness remains a separate question.

Why post-synthesis and LLM-generated checkers? Holding the checker fixed attributes improvement to an edit rather than a new generation draw. The interface can also accept a compatible handwritten checker and paired target, although that population and evidence-assisted fresh generation remain unevaluated.

Evidence as context compression. Provenance-tagged, patch-scoped views bound the editor’s context instead of exposing full repository and analyzer dumps. This frames refinement as specification completion rather than report suppression [11, 33, 35, 36, 45]; the present study does not isolate compression efficacy.

Backend complementarity. The CSA study and separate E2 CodeQL replay exercise two detector formats. They do not show that every evidence role was populated from an analyzer-internal representation. In the audited CSA cohort, the verified native interface is a CFG/call-graph dump; source-derived and diagnostic records remain separately labelled. A CodeQL-native coverage claim requires retained database-query provenance.

Failure modes. The dominant failure mode is over-pruning: a candidate suppresses fixed-side warnings by deleting or over-constraining the vulnerable trigger. PDS rejects such cases as validity loss, never success. Other modes include evidence insufficiency (a collector misses the needed fact), backend mismatch (the analyzer cannot expose the required relation), and model-side edit quality (the model cannot turn a correct fact into compilable code). In each case the gate rejects the candidate rather than silently accepting it.

Limited semantic generalization. SEMWEAVER validates patch-local discrimination [8, 22, 38, 39]. The motivating checker uses array extent and index bounds, not patch names or statement

adjacency. It replays correctly on the original Linux pair and passes separate development fixtures varying names, statements, capacities, and supported unit-step loop/guard forms. This is not held-out vulnerability-family recall; arbitrary aliasing and dynamic extents remain outside scope.

Precision and scaling boundary. The matched 15-case comparison retains all vulnerable-side warning signals with 60 fixed-object alerts versus KNighter’s 72; the full-cohort GPT run additionally makes five previously silent vulnerable revisions warning-positive. Evidence replay completes for all 39 with one native-evidence abstention. This supports bounded automated processing, not whole-kernel throughput or independently verified target recall. The full-cohort ablation tie prevents assigning the system gain to internal records alone.

8 THREATS TO VALIDITY

Internal. An error in evidence collection, LLM editing, or paired validation could produce a spurious refinement. The gate rejects candidates that lose the vulnerable-side warning signal, fail compilation, or lack a valid paired execution, but cannot establish generalization outside the observed scope. One GPT candidate’s generated helper did not terminate during paired execution; it was excluded as an invalid candidate and the original checker retained, not relabelled as a successful edit or a missed target. Independent checker replay and raw-output/source-scope hashes reduce, but do not eliminate, implementation and oracle risks. The 39 starting checkers and all scored refinements are automatic; earlier manual completions and mixed-provenance cross-backend rows are excluded. Repeated decodes reuse the same development-informed subset and do not constitute fresh-subject confirmation. The full-cohort ablation is tied, and its one abstention prevents treating internal evidence as universally available.

Construct. PDS evaluates warning discrimination between paired revisions under a patch-related-object scope. It does not establish that every vulnerable-side warning detects the target bug; G08 supplies a concrete counterexample. The secondary F_1 uses 78 paired patch-version decisions, not adjudicated warning sites, and must not be interpreted as target-vulnerability recall or deployment precision. Fixed-side alerts measure inspection burden, not a complete false-positive oracle. Fixed-side silence is enforced over patch-related build objects rather than just the changed line, but a checker can still overfit the patch function.

External. The reported 39-case cohort is a frozen, one-per-commit selection from 286 upstream generated records, not a random or full-population screen. Ranking candidates by upstream TP/TN metadata may favor certain starting checkers, although independent paired replay supplies all reported labels; no effect rate over all 286 is claimed. The 15 fixed-noisy cases are the complete observed warning-both stratum within this selected cohort, while the other 24 have a distinct warning-recovery task. KNighter’s no-report branch does not edit those 24, so the full-system comparison must not be read as same-task recovery superiority. Model configurations have different output ceilings, and their aliases do not identify immutable model revisions; the results describe the evaluated configurations. Refined detectors are validated for patch-local warning discrimination, not unseen code or future kernel versions. The motivating example’s perturbation suites are development diagnostics, not held-out robustness estimates.

9 RELATED WORK

Static Analysis and Checker Languages. Static-analysis systems provide the substrate that SEMWEAVER refines against. Path-sensitive analyzers, abstract interpreters, and symbolic engines expose guards, states, paths, and diagnostics reusable as evidence [3, 6, 12, 14] and deploy at industrial scale [4, 7, 41]. Declarative languages offer a complementary approach by querying program facts through relational rules and points-to specifications [2, 5, 20], while semantic patching encodes recurring transformations over systems code [37]. SEMWEAVER introduces no new analyzer

or query language. It treats LLM-generated analyzer artifacts as editable targets and uses analyzer-native facts as evidence for semantic completion.

Vulnerability Detection and Patch Evidence. Static vulnerability detection has long used taint and data-flow reasoning [1, 21, 30], graph and AST representations [46, 47], and learning-based token-, graph-, slice-, and line-level models [16, 28, 29, 40], though datasets and evaluations show that semantic robustness remains difficult [8, 15]. Clone- and patch-driven systems use historical fixes as compact witnesses [19, 24, 27]. These motivate our use of patches as specifications, but SEMWEAVER targets a different artifact: an already generated checker whose fixed-side behavior reveals missing semantics.

LLM-Based Code and Security Reasoning. Code LLMs can synthesize programs [10], but security studies caution against trusting generated artifacts without validation [38, 39]. SEMWEAVER edits a static-analysis artifact with an LLM; paired analyzer executions, rather than the model’s explanation, determine acceptance.

Checker Synthesis and Feedback-Guided Repair. Patch-driven checker synthesis, especially KNIGHTER [48], is the closest prior work. LLMs transform historical security fixes into specialized analyzers that compile, execute, and are reused. SEMWEAVER is complementary because it assumes such a detector exists and refines it post-synthesis. A generator’s validation feedback reveals which side a detector fires on but not whether the missing predicate is a guard, barrier, flow, state, lifetime, type filter, or API-usage relation.

The loop also relates to automated program repair, synthesis, and counterexample-guided workflows that edit candidates under compiler, test, verifier, symbolic, or learned feedback [11, 23, 25, 32, 33, 35, 36, 45], and to LLM refinement and agent work on iterative feedback, tool use, and deliberative search [34, 42, 44, 49, 50]. SEMWEAVER differs in artifact and acceptance: the edit target is a detector, not the vulnerable program, and acceptance requires paired-version warning discrimination rather than compilation or fewer warnings alone. It rejects suppression of all vulnerable-side warnings, without independently proving target detection.

Analyzer Context for LLMs and Alert Triage. Prior work already uses analysis information in LLM workflows. Chapman et al. interleave static-analysis intermediate results and LLM prompts to infer error specifications [9]; IRIS infers taint specifications and performs contextual analysis around CodeQL results [26]. LLM4PFA reasons about interprocedural path feasibility to filter static bug reports [13], while ZeroFalse enriches alerts with flow-sensitive traces before LLM adjudication [18]. We therefore do not claim to be the first to prompt an LLM with analyzer information. The narrower distinction is the edited artifact and gate: SEMWEAVER changes an existing generated checker, and accepts that code change only if paired patch-revision validation retains a vulnerable-side warning and silences the fixed side. Even this distinction is useful only if the evidence origin and checker outcome are independently auditable.

10 CONCLUSION

We presented SEMWEAVER, an automated post-synthesis checker editor with provenance-labelled analyzer context and a paired-warning gate. On 39 frozen generated-only CSA checkers, its GPT configuration reaches seven patch-discriminating successes versus one for KNIGHTER’s actual loop; on the directly matched 15 fixed-noisy cases it retains 60 fixed-side object alerts versus KNIGHTER’s 72 while keeping all vulnerable revisions warning-positive. The full-cohort no-internal ablation ties the native condition, so an incremental causal benefit from internal records is not established here. The evidence supports bounded patch-local warning-noise reduction and warning recovery, not independently adjudicated target recall or unseen-site vulnerability generalization.

DATA AVAILABILITY

The anonymized repository is at <https://anonymous.4open.science/r/SemWeaver-FD1E/>. Its English replication package contains the implementation, 39 frozen generated-only checker and patch inputs, Clang raw-output/provenance bindings, the actual KNighter and SEMWEAVER outputs, model exchanges, repeated-subset results, and an offline verifier. Historical manual and mixed-provenance materials are excluded from the scored results. Full reruns require public Linux revisions and the pinned toolchain; the curated package is intended to remain public upon acceptance.

REFERENCES

- [1] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Oteau, and Patrick McDaniel. 2014. FlowDroid: Precise Context, Flow, Field, Object-Sensitive and Lifecycle-Aware Taint Analysis for Android Apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 259–269. <https://doi.org/10.1145/2594291.2594299>
- [2] Pavel Avgustinov, Oege de Moor, Michael Peyton Jones, and Max Schäfer. 2016. QL: Object-Oriented Queries on Relational Data. In *Proceedings of the 30th European Conference on Object-Oriented Programming (ECOOP)*. 2:1–2:25. <https://doi.org/10.4230/LIPIcs.ECOOP.2016.2>
- [3] Thomas Ball and Sriram K. Rajamani. 2002. The SLAM Project: Debugging System Software via Static Analysis. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 1–3. <https://doi.org/10.1145/503272.503274>
- [4] Al Bessey, Ken Block, Ben Chelf, Andy Chou, Bryan Fulton, Seth Hallem, Charles Henri-Gros, Asya Kamsky, Scott McPeak, and Dawson Engler. 2010. A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World. *Commun. ACM* 53, 2 (2010), 66–75. <https://doi.org/10.1145/1646353.1646374>
- [5] Martin Bravenboer and Yannis Smaragdakis. 2009. Strictly Declarative Specification of Sophisticated Points-to Analyses. In *Proceedings of the 24th ACM SIGPLAN Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*. 243–262. <https://doi.org/10.1145/1640089.1640108>
- [6] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 209–224. https://www.usenix.org/legacy/event/osdi08/tech/full_papers/cadar/cadar.pdf
- [7] Cristiano Calcagno, Dino Distefano, Jérémy Dubreil, Dominik Gabi, Pieter Hooimeijer, Martino Luca, Peter O’Hearn, Irene Papakonstantinou, Jim Purbrick, and Dulma Rodriguez. 2015. Moving Fast with Software Verification. In *Proceedings of the 7th NASA Formal Methods Symposium (NFM)*. 3–11. https://doi.org/10.1007/978-3-319-17524-9_1
- [8] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2022. Deep Learning Based Vulnerability Detection: Are We There Yet? *IEEE Transactions on Software Engineering* 48, 9 (2022), 3280–3296. <https://doi.org/10.1109/TSE.2021.3087402>
- [9] Patrick J. Chapman, Cindy Rubio-González, and Aditya V. Thakur. 2024. Interleaving Static Analysis and LLM Prompting. In *Proceedings of the 13th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis*. 9–17. <https://doi.org/10.1145/3652588.3663317>
- [10] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). <https://doi.org/10.48550/arXiv.2107.03374>
- [11] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. 2000. Counterexample-Guided Abstraction Refinement. In *Proceedings of the 12th International Conference on Computer Aided Verification (CAV)*. 154–169. https://doi.org/10.1007/10722167_15
- [12] Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, David Monniaux, and Xavier Rival. 2005. The ASTRÉE Analyzer. In *Proceedings of the 14th European Symposium on Programming (ESOP)*. 21–30. https://doi.org/10.1007/978-3-540-31987-0_3
- [13] Xueying Du, Kai Yu, Chong Wang, Yi Zou, Wentai Deng, Zuoyu Ou, Xin Peng, Lingming Zhang, and Yiling Lou. 2025. Minimizing False Positives in Static Bug Detection via LLM-Enhanced Path Feasibility Analysis. *arXiv:2506.10322* <https://arxiv.org/abs/2506.10322>
- [14] Dawson Engler, David Yu Chen, Seth Hallem, Andy Chou, and Benjamin Chelf. 2001. Bugs as Deviant Behavior: A General Approach to Inferring Errors in Systems Code. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*. 57–72. <https://doi.org/10.1145/502034.502041>

- [15] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*. 508–512. <https://doi.org/10.1145/3379597.3387501>
- [16] Michael Fu and Chakkrit Tantithamthavorn. 2022. LineVul: A Transformer-Based Line-Level Vulnerability Prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*. 608–620. <https://doi.org/10.1145/3524842.3528452>
- [17] GitHub. 2026. CodeQL Documentation. <https://codeql.github.com/docs/>. Accessed: Jun. 27, 2026.
- [18] Mohsen Iranmanesh, Sina Moradi Sabet, Sina Marefat, Ali Javidi Ghasr, Allison Wilson, Iman Sharafaldin, and Mohammad A. Tayebi. 2025. ZeroFalse: Improving Precision in Static Analysis with LLMs. arXiv:2510.02534 <https://arxiv.org/abs/2510.02534>
- [19] Jiyong Jang, Abeer Agrawal, and David Brumley. 2012. ReDeBug: Finding Unpatched Code Clones in Entire OS Distributions. In *Proceedings of the 2012 IEEE Symposium on Security and Privacy*. 48–62. <https://doi.org/10.1109/SP.2012.13>
- [20] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. 2016. Soufflé: On Synthesis of Program Analyzers. In *Proceedings of the 28th International Conference on Computer Aided Verification (CAV)*. 422–430. https://doi.org/10.1007/978-3-319-41540-6_23
- [21] Nenad Jovanovic, Christopher Kruegel, and Engin Kirda. 2006. Pixy: A Static Analysis Tool for Detecting Web Application Vulnerabilities. In *Proceedings of the 2006 IEEE Symposium on Security and Privacy*. 258–263. <https://doi.org/10.1109/SP.2006.29>
- [22] Raphaël Khoury, Anderson R. Avila, Jacob Brunelle, and Baba Mamadou Camara. 2023. How Secure Is Code Generated by ChatGPT?. In *Proceedings of the 2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 2445–2451. <https://doi.org/10.1109/SMC53992.2023.10394237>
- [23] Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. 2013. Automatic Patch Generation Learned from Human-Written Patches. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 802–811. <https://doi.org/10.1109/ICSE.2013.6606626>
- [24] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. 2017. VUDDY: A Scalable Approach for Vulnerable Code Clone Discovery. In *Proceedings of the 2017 IEEE Symposium on Security and Privacy*. 595–614. <https://doi.org/10.1109/SP.2017.62>
- [25] Claire Le Goues, Michael Dewey-Vogt, Stephanie Forrest, and Westley Weimer. 2012. A Systematic Study of Automated Program Repair: Fixing 55 out of 105 Bugs for \$8 Each. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*. 3–13. <https://doi.org/10.1109/ICSE.2012.6227211>
- [26] Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/582d4e27fa24168f3af1f4582655034b-Abstract-Conference.html
- [27] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Hanchao Qi, and Jie Hu. 2016. VulPecker: An Automated Vulnerability Detection System Based on Code Similarity Analysis. In *Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC)*. 201–213. <https://doi.org/10.1145/2991079.2991102>
- [28] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2022. SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* 19, 4 (2022), 2244–2258. <https://doi.org/10.1109/TDSC.2021.3051525>
- [29] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. In *Proceedings of the 25th Network and Distributed System Security Symposium (NDSS)*. <https://doi.org/10.14722/ndss.2018.23158>
- [30] Benjamin Livshits and Monica S. Lam. 2005. Finding Security Vulnerabilities in Java Applications with Static Analysis. In *Proceedings of the 14th USENIX Security Symposium*. 271–286. https://www.usenix.org/legacy/events/sec05/tech/full_papers/livshits/livshits.pdf
- [31] LLVM Project. 2026. Clang Static Analyzer Documentation. <https://clang-analyzer.llvm.org/>. Accessed: Jun. 27, 2026.
- [32] Fan Long and Martin Rinard. 2015. Staged Program Repair with Condition Synthesis. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 166–178. <https://doi.org/10.1145/2786805.2786811>
- [33] Fan Long and Martin Rinard. 2016. Automatic Patch Generation by Learning Correct Code. In *Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 298–312. <https://doi.org/10.1145/2837614.2837617>
- [34] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 46534–46594. https://proceedings.neurips.cc/paper_files/paper/2023/file/91edff07232fb1b55a505a9e9f6c0ff3-Paper-Conference.pdf

- [35] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. 2016. Angelix: Scalable Multiline Program Patch Synthesis via Symbolic Analysis. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. 691–701. <https://doi.org/10.1145/2884781.2884807>
- [36] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. 2013. SemFix: Program Repair via Semantic Analysis. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 772–781. <https://doi.org/10.1109/ICSE.2013.6606623>
- [37] Yoann Padioleau, Julia L. Lawall, and Gilles Muller. 2008. Documenting and Automating Collateral Evolutions in Linux Device Drivers. In *Proceedings of the 3rd ACM SIGOPS/EuroSys European Conference on Computer Systems*. 247–260. <https://doi.org/10.1145/1352592.1352618>
- [38] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy*. 754–768. <https://doi.org/10.1109/SP46214.2022.9833571>
- [39] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2785–2799. <https://doi.org/10.1145/3576915.3623157>
- [40] Rebecca Russell, Louis Kim, Lei Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul Ellingwood, and Marc McConley. 2018. Automated Vulnerability Detection in Source Code Using Deep Representation Learning. In *Proceedings of the 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*. 757–762. <https://doi.org/10.1109/ICMLA.2018.00120>
- [41] Caitlin Sadowski, Jeffrey van Gogh, Ciera Jaspan, Emma Soderberg, and Collin Winter. 2015. Tricorder: Building a Program Analysis Ecosystem. In *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE)*. 598–608. <https://doi.org/10.1109/ICSE.2015.76>
- [42] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 68539–68551. https://proceedings.neurips.cc/paper_files/paper/2023/file/d842425e4bf79ba039352da0f658a906-Paper-Conference.pdf
- [43] Srinivasan Shanmugam. 2024. drm/amd/display: Fix buffer overflow in get_host_router_total_dp_tunnel_bw(). Linux kernel commit 97cba232549b9fe7e491fb60a69cf93075015f29. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=97cba232549b9fe7e491fb60a69cf93075015f29> Commit metadata and source checked locally, September 27, 2026.
- [44] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 8634–8652. https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf
- [45] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 404–415. <https://doi.org/10.1145/1168857.1168907>
- [46] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and Discovering Vulnerabilities with Code Property Graphs. In *Proceedings of the 2014 IEEE Symposium on Security and Privacy*. 590–604. <https://doi.org/10.1109/SP.2014.44>
- [47] Fabian Yamaguchi, Markus Lottmann, and Konrad Rieck. 2012. Generalized Vulnerability Extrapolation Using Abstract Syntax Trees. In *Proceedings of the 28th Annual Computer Security Applications Conference (ACSAC)*. 359–368. <https://doi.org/10.1145/2420950.2421003>
- [48] Chenyuan Yang, Zijie Zhao, Zichen Xie, Haoyu Li, and Lingming Zhang. 2025. KNighter: Transforming Static Analysis with LLM-Synthesized Checkers. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*. 655–669. <https://doi.org/10.1145/3731569.3764827>
- [49] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 11809–11822. https://proceedings.neurips.cc/paper_files/paper/2023/file/271db9922b8d1f4dd7aaef84ed5ac703-Paper-Conference.pdf
- [50] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synnergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=WE_vluYUL-X